Speech Analysis: Identification and Detection of Voiced Consonants

VE216 Project Topic 2
Python implementation as a MATLAB substitute

April 2026

Abstract

This project studies the spectrum of voiced consonants and implements automatic detection of selected consonant letters in spoken command words. Four primary voiced consonants, /d/, /g/, /l/, and /n/, are analyzed in detail; /r/ and /j/ are included as extra voiced targets and two non-target words are grouped as an "other" class. Because MATLAB is unavailable locally, the full signal-processing pipeline is implemented in Python with NumPy, SciPy, and scikit-learn. The real audio data come from the TensorFlow mini Speech Commands data set, containing 8000 one-second utterances from eight command words. A classical detector based on short-time spectra, MFCCs, and nearest-centroid matching is compared with an AI detector implemented as a small multilayer perceptron. On a held-out 2000-file test set, the classical detector reaches 52.25% consonant-class accuracy, while the AI detector reaches 76.30%. The experiment confirms that voiced consonants are visible in short-time spectral features, but robust detection benefits from data-driven nonlinear classification.

1 Introduction

The project statement asks for spectral analysis of at least four voiced consonant letters and a program that detects the presence of selected letters in spoken words. The target is not full speech recognition; it is a smaller signal-processing task: use the spectrum of consonants to identify whether a word contains a particular voiced consonant cue.

A voiced consonant has periodic glottal excitation during articulation, so its spectrum often contains harmonic structure and low-frequency voicing energy. However, consonants differ by manner and place of articulation. Stops such as /d/ and /g/ contain short transient bursts after closure release, nasals such as /n/ contain low-frequency resonances and antiresonances, and liquids such as /l/ and /r/ have formant-like spectral patterns. Therefore, the relevant representation should preserve both time localization and spectral shape.

This report follows a journal-style workflow:

1. define a real-data consonant detection problem;
2. compute short-time spectral and cepstral features;
3. visualize representative spectra for the target consonants;
4. implement a classical nearest-centroid detector;
5. implement an AI detector and compare the two methods.

2 Data set and problem setting

2.1 Real data

The experiment uses the TensorFlow mini Speech Commands data set from the official TensorFlow audio tutorial. It is a subset of the Speech Commands data set introduced by Warden for

limited-vocabulary speech recognition. The local data used in this project contain 8000 WAV files, sampled at 16 kHz, with 1000 examples for each of the following words:

`down`, `go`, `left`, `no`, `right`, `yes`, `stop`, `up`.

The words are mapped to their initial consonant targets as shown in Table 1. The project requires at least four letters, so /d/, /g/, /l/, and /n/ are the primary targets. /r/ and /j/ are included as additional voiced consonantal classes. `stop` and `up` are used as non-target examples and merged into an `other` class.

Table 1: Word-to-consonant mapping used by the detector.

Word	Target class	Role
`down`	/d/	primary voiced stop
`go`	/g/	primary voiced stop
`left`	/l/	primary voiced liquid
`no`	/n/	primary voiced nasal
`right`	/r/	extra voiced liquid
`yes`	/j/	extra voiced glide
`stop`	other	non-target initial /s/
`up`	other	vowel-initial non-target

This mapping is a controlled simplification. The detector classifies the most salient initial consonant of short command words, rather than performing phone boundary segmentation inside long continuous speech. This is still a meaningful version of the project task because the program detects whether a selected consonant cue is present in a spoken word.

2.2 Train-test split

The data set is split at file level into 6000 training files and 2000 held-out testing files, stratified by target class. The split is fixed with random seed 216 for reproducibility. The final class counts are:

Class	/d/	/g/	/l/	/n/	/r/	/j/	other
Files	1000	1000	1000	1000	1000	1000	2000

3 Signal-processing method

3.1 Preprocessing

Each WAV file is converted to mono, normalized to unit peak amplitude, and padded or cropped to one second. A pre-emphasis filter is applied:

$$y[n] = x[n] - 0.97x[n-1]. \tag{1}$$

The signal is then divided into 25 ms Hamming-windowed frames with a 10 ms hop. This is a standard short-time assumption: speech is nonstationary over long intervals, but approximately stationary inside a short frame.

3.2 Short-time spectrum

For a frame $x_m[n]$, the short-time Fourier transform is

$$X_m[k] = \sum_{n=0}^{N-1} x_m[n]w[n]e^{-j2\pi kn/N}. \tag{2}$$

The spectrogram uses $20\log_{10}|X_m[k]|$. It shows how consonant energy is distributed over time and frequency. This is especially useful for distinguishing stops, nasals, and liquids because their acoustic cues are not equally visible in a full-word average spectrum.

3.3 MFCCs and classical spectral descriptors

The main feature representation is a combination of MFCCs and interpretable spectral descriptors. For each frame, the power spectrum is passed through 40 triangular mel filters. The log mel energies are transformed by a discrete cosine transform:

$$c_q = \sum_{m=1}^{M} \log E_m \cos\left[\frac{\pi q}{M}(m-0.5)\right], \qquad q = 0, \ldots, 12. \tag{3}$$

The first 13 MFCCs are kept. MFCCs are widely used because they summarize the spectral envelope in a way that roughly matches human auditory frequency resolution.

Additional descriptors are computed from each frame:

$$\text{centroid} = \frac{\sum_k f_k|X[k]|}{\sum_k |X[k]|}, \tag{4}$$

$$\text{bandwidth} = \sqrt{\frac{\sum_k |X[k]|(f_k - \text{centroid})^2}{\sum_k |X[k]|}}, \tag{5}$$

$$\text{rolloff}_{0.85} = \min f_i \quad \text{s.t.} \quad \sum_{k \le i} |X[k]| \ge 0.85 \sum_k |X[k]|. \tag{6}$$

RMS energy and zero-crossing rate are also included. The final feature vector concatenates mean and standard deviation statistics with the fixed-length MFCC time sequence. The statistical part supports interpretable comparison; the time-sequence part gives the AI model access to temporal order.

4 Implemented detectors

4.1 Classical nearest-centroid detector

The classical baseline uses the same handcrafted features but no neural network. After standardization, each target class is represented by its training-set centroid. A test utterance is assigned to the class with the largest cosine similarity:

$$\hat{y} = \arg\max_c \frac{z^\top \mu_c}{\|z\|_2 \|\mu_c\|_2}, \tag{7}$$

where z is the standardized feature vector and μ_c is the centroid for class c.

Listing 1: Classical detector pseudocode.

```
for each training utterance:
    compute STFT, MFCC, spectral statistics
standardize all feature vectors
```

```
for each consonant class c:
    centroid[c] = mean(feature vectors with label c)
for each test utterance z:
    predict class with maximum cosine_similarity(z, centroid[c])
```

This detector is transparent: its decision is based on similarity to average spectral templates. Its weakness is that it cannot easily model nonlinear speaker variation, timing variation, or overlapping cues.

4.2 AI detector

The AI method uses a multilayer perceptron implemented by scikit-learn. The input feature vector is the same standardized MFCC/spectral representation. The network has two hidden layers of 64 and 32 ReLU units and is trained by backpropagation. It redoes the detection step with a nonlinear decision boundary:

$$\hat{y} = \mathrm{softmax}(W_3\sigma(W_2\sigma(W_1 z + b_1) + b_2) + b_3). \tag{8}$$

This model is still small enough to explain and reproduce. It is not a large speech-recognition system; it is an AI classifier for the same voiced-consonant detection task.

5 Results

5.1 Spectral analysis

Figure 1 shows representative waveforms and spectrograms for the four primary voiced consonants. The stop consonants /d/ and /g/ are more transient: their cues are concentrated near the word onset and release. The liquid /l/ and nasal /n/ show more sustained low-frequency voiced structure. These observations justify using short-time features instead of only one global Fourier transform for the whole word.

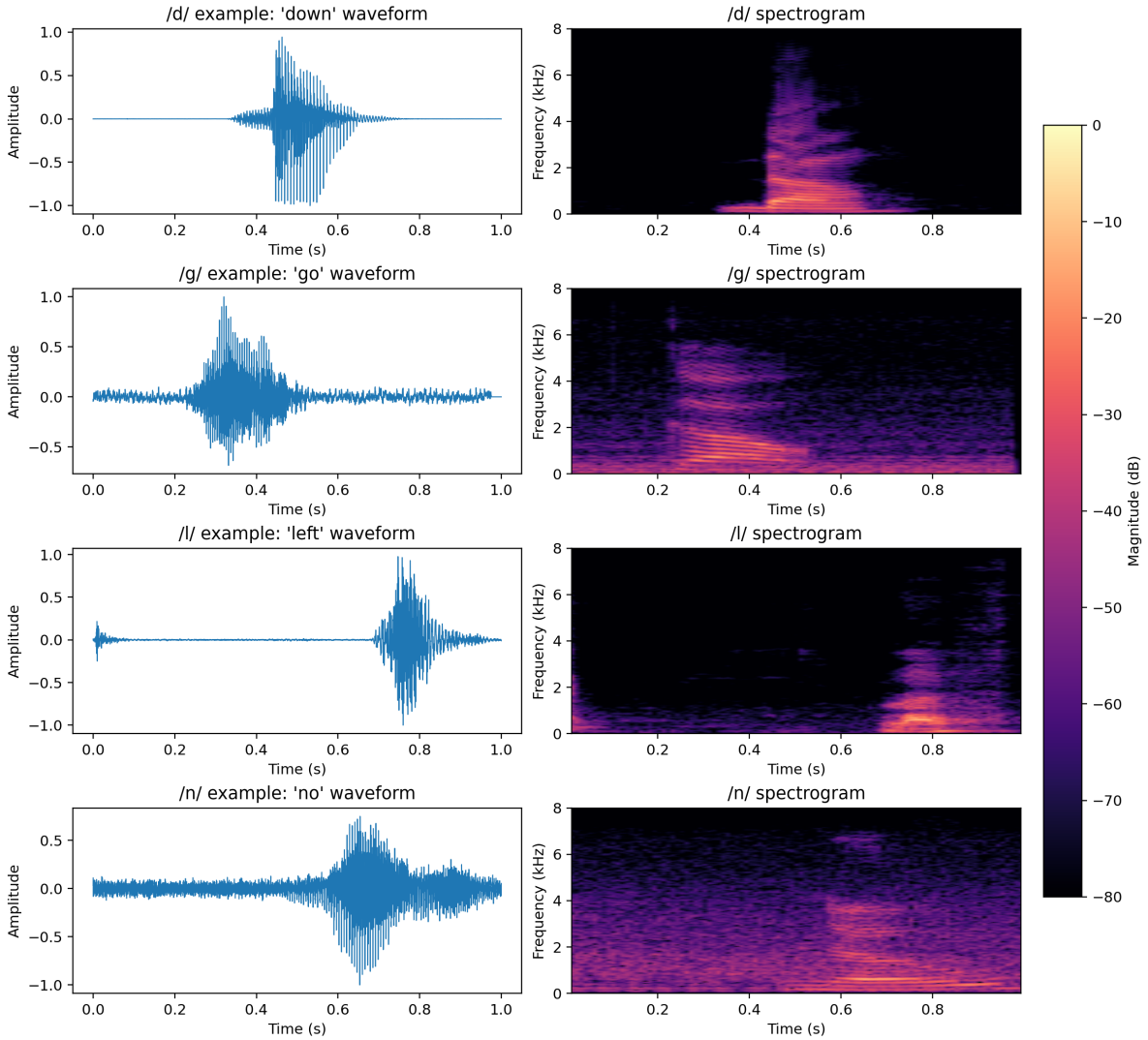


Figure 1: Waveforms and spectrograms for representative /d/, /g/, /l/, and /n/ examples from the real data set.

Figure 2 compares representative short-time average spectra for all voiced target classes. The curves are not perfectly separable by one frequency band, which explains why a simple single-threshold detector is insufficient. The task requires a combination of spectral envelope, energy, and temporal features.

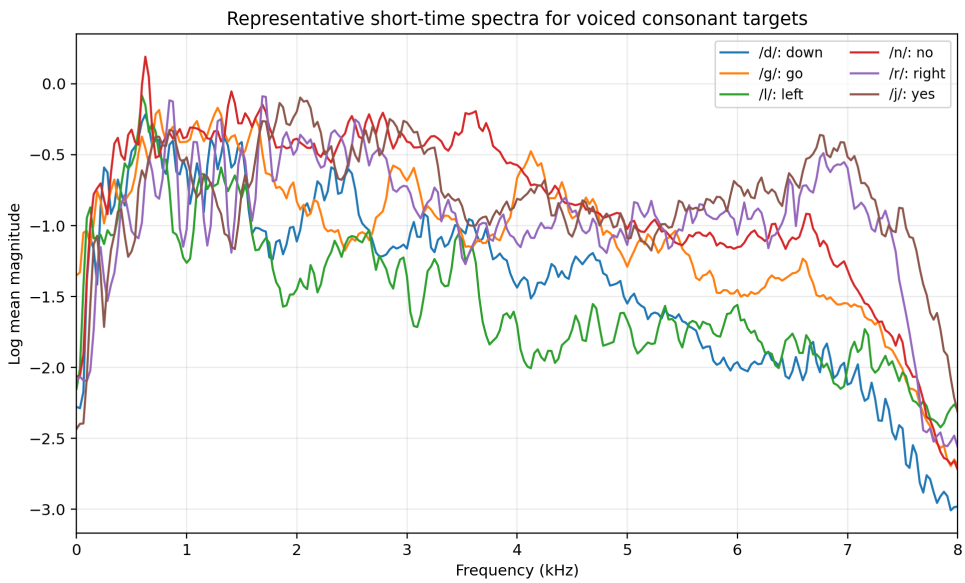


Figure 2: Representative short-time spectra for six voiced consonant targets.

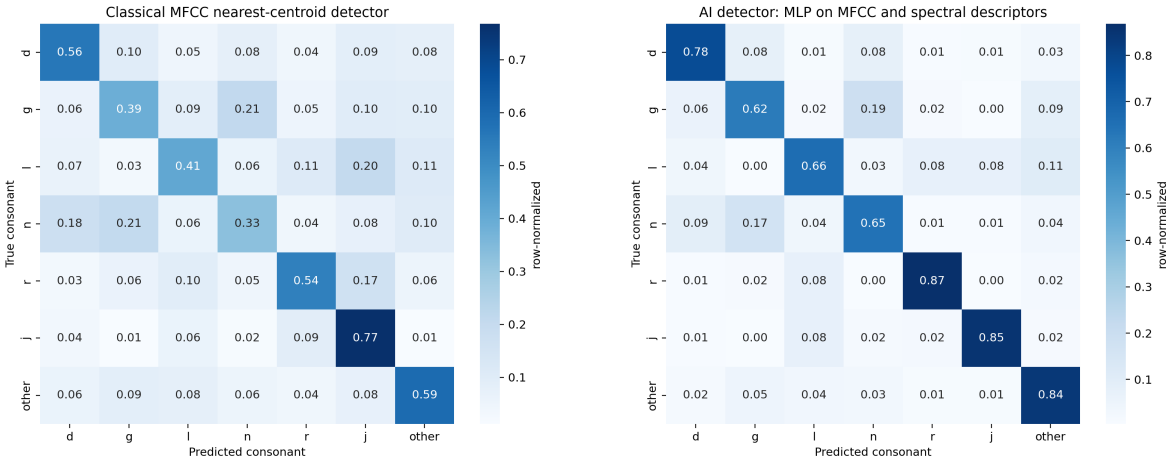
5.2 Detection accuracy

Table 2 summarizes held-out accuracy. The AI detector improves accuracy by 24.05 percentage points over the classical centroid baseline.

Table 2: Held-out consonant-class detection accuracy.

Method	Test files	Accuracy
Classical MFCC nearest-centroid detector	2000	52.25%
AI detector: MLP on MFCC and spectral descriptors	2000	76.30%

Figure 3 gives normalized confusion matrices. The classical method confuses several voiced classes because each centroid is only an average spectral template. The AI method is substantially stronger, especially for /d/, /r/, /j/, and the non-target class. The hardest primary class remains /n/, which is often confused with other voiced sonorants or with the broad `other` class.



(a) Classical detector

(b) AI detector

Figure 3: Row-normalized confusion matrices on the 2000-file test set.

5.3 Binary detection of the four required letters

The project statement asks for detecting the presence of certain letters. Table 3 reports one-vs-rest metrics for the four required voiced consonants. The AI detector improves all four F1 scores.

Table 3: One-vs-rest detection metrics for the four primary voiced consonants.

Target	Method	Precision	Recall	F1
/d/	Classical	0.526	0.560	0.543
/d/	AI MLP	0.755	0.776	0.765
/g/	Classical	0.396	0.388	0.392
/g/	AI MLP	0.628	0.620	0.624
/l/	Classical	0.446	0.412	0.428
/l/	AI MLP	0.695	0.664	0.679
/n/	Classical	0.381	0.328	0.353
/n/	AI MLP	0.623	0.648	0.635

6 Discussion

6.1 What the classical method captures

The classical method uses exactly the kind of spectrum-based reasoning expected in a signals-and-systems project. STFT localizes consonant cues in time, MFCCs describe the spectral envelope, and centroid/bandwidth/rolloff summarize frequency distribution. These features are interpretable and can be inspected directly from figures.

The main limitation is that a single centroid per consonant cannot represent all speakers, accents, microphone conditions, and timing differences. For example, the /g/ in `go` may vary strongly by speaker and vowel transition, and the /n/ in `no` is influenced by the following vowel. This reduces the effectiveness of pure template matching.

6.2 What the AI method improves

The MLP uses the same physical features but learns nonlinear combinations of them. This is why it outperforms the nearest-centroid detector. The result does not mean that AI replaces signal processing. In this experiment, AI works because the signal-processing front end provides a compact representation: normalization, framing, STFT, mel filtering, and MFCC extraction still define the information that the model sees.

6.3 Limitations

This project is intentionally scoped to be reproducible within the VE216 deadline. It has three limitations:

1. The labels are word-level initial consonant labels, not manually segmented phone labels.
2. The data set contains command words, not arbitrary words such as `book` or `boy`.
3. The AI model is a small classifier, not a full automatic speech recognition model.

These limitations can be removed in future work by using a phonetically aligned corpus such as TIMIT or by manually segmenting consonant intervals in recorded words.

7 Conclusion

The project implemented a complete voiced-consonant analysis and detection pipeline with real speech data. Four required voiced consonants, /d/, /g/, /l/, and /n/, were analyzed with waveforms, spectrograms, and short-time spectra. A classical MFCC nearest-centroid detector and a small AI detector were implemented and compared. The AI detector achieved 76.30% test accuracy, compared with 52.25% for the classical detector, and improved one-vs-rest F1 scores for all four required consonants. The main conclusion is that spectral analysis provides the necessary acoustic representation, while data-driven classification improves robustness under speaker and recording variability.

Reproducibility

The implementation files are organized as follows:

- `src/download_data.sh`: downloads the real audio data set.
- `src/run_experiment.py`: extracts features, trains detectors, saves metrics, and generates figures.
- `build/results/summary.json`: final test accuracy and data counts.
- `build/results/binary_detection_metrics.csv`: one-vs-rest metrics for /d/, /g/, /l/, and /n/.
- `figures/`: generated figures included in this report.

The main commands are:

```
python -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
src/download_data.sh
.venv/bin/python src/run_experiment.py
latexmk -xelatex -interaction=nonstopmode -halt-on-error report.tex
```

References

[1] P. Warden, "Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition," arXiv:1804.03209, 2018. [Online]. Available: https://arxiv.org/abs/1804.03209

[2] TensorFlow, "Simple audio recognition: Recognizing keywords," official tutorial. [Online]. Available: https://www.tensorflow.org/tutorials/audio/simple_audio

[3] S. B. Davis and P. Mermelstein, "Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 28, no. 4, pp. 357–366, 1980. doi: 10.1109/TASSP.1980.1163420.

[4] SciPy Developers, "scipy.signal.stft," SciPy documentation. [Online]. Available: https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.stft.html

[5] scikit-learn Developers, "sklearn.neural_network.MLPClassifier," scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html

[6] B. McFee et al., "librosa: Audio and Music Signal Analysis in Python," 2015. [Online]. Available: https://librosa.org/doc/latest/feature.html